

Implement Client Script & UI Policy (Incident)

Phase 3: Project Design – Problem-Solution Fit

Item	Details
Date	September 30, 2026
Team ID	SWTID-2026-6974
Project Name	Implement Client Script & UI Policy (Incident)
Platform	ServiceNow
Maximum Marks	2 Marks

1. Problem-Solution Fit Template

1.1 Purpose

The Problem-Solution Fit template checks that the solution addresses a real problem for a defined group of users. It helps to:

- Understand the situation in which the problem occurs and the people it affects.
- Confirm that the chosen solution fits the users' behaviour, constraints and triggers.
- Reduce the risk of building controls that users ignore, bypass or find unhelpful.
- Give the team a shared reference for design decisions in later phases.

1.2 Problem Summary

Incident records often reach the system with missing or inconsistent data. High-impact incidents can be saved without an assignee, Urgency can disagree with Impact, and State can be changed directly from the list view. This affects triage, routing, SLA compliance and report accuracy.

2. Template Elements

Element	Description
1. Customer Segment(s)	Service desk agents, incident managers and fulfiller users who create and update Incident records; ServiceNow administrators who maintain the configuration.
2. Jobs-to-be-Done / Problems	Log incidents quickly and correctly; route high-impact incidents to the right owner; keep Impact, Urgency and State consistent; produce reliable reports.
3. Triggers	A high-impact incident is raised; an SLA is at risk; a report shows incomplete or inconsistent records; a manager finds an incident with no owner.
4. Emotions: Before / After	Before: rushed, uncertain, anxious about missed SLAs and rework. After: confident, guided by clear messages, and assured that records are complete.
5. Available Solutions	Manual checks, agent training and checklists; server-side Business Rules; Data Policies; ad hoc reviews of incident quality.
6. Customer Constraints	Time pressure while logging; limited tolerance for extra steps; no wish to lose flexibility on lower-impact incidents; varied experience levels among agents.
7. Behaviour	Fill only the fields that seem necessary, change Impact and Urgency independently, edit State from the list to save time, and return later to

Element	Description
	correct records.
8. Channels of Behaviour (Online)	The ServiceNow Incident form and the Incident list view in the browser.
9. Channels of Behaviour (Offline)	Team briefings, shared checklists, and verbal handovers between agents and managers.
10. Problem Root Cause	The form does not enforce business rules at the point of entry, so correctness depends on manual awareness and memory.
11. Your Solution	Use a UI Policy and UI Policy Actions to control field behaviour, and Client Scripts (onChange, onSubmit, onCellEdit) to automate Urgency, validate on save and block list edits of State.

3. Mapping Problems to Solutions

No.	Problem	Solution Component	Type
1	High-impact incident has no assignment group	High Impact Control with Assignment group set to mandatory	UI Policy + Action
2	Urgency edited independently of Impact	Urgency set to read-only when Impact is High	UI Policy Action
3	Urgency not aligned with Impact	Auto set urgency for high impact	onChange Client Script
4	High-impact incident saved without an assignee	Prevent save if Assigned To missing	onSubmit Client Script
5	State changed from the list view	Prevent state change via list edit	onCellEdit Client Script
6	Restrictions remain after Impact is lowered	Reverse if false on the UI Policy	UI Policy setting

4. Fit Analysis

4.1 Why the Solution Fits

- **Right place:** Rules run inside the form, where the problem occurs, so users get feedback immediately.
- **Right scope:** Rules apply only when Impact is High and reverse automatically, so normal incidents stay flexible.
- **Low effort for users:** Urgency is set automatically, reducing manual effort for agents.
- **Simple to build:** The solution uses standard ServiceNow features with no custom tables or integrations.

4.2 Comparison with Available Solutions

Alternative	Limitation	How This Solution Improves It
Manual checks and training	Depends on memory and discipline; inconsistent	Enforces rules automatically at entry
Server-side Business Rule	Feedback appears only after save; more effort	Immediate, field-level feedback on the form
Data Policy	Less control over field messages	Custom messages and list-edit

Alternative	Limitation	How This Solution Improves It
	and list edits	control
Post-entry review	Errors found late, causing rework	Errors prevented before submission

4.3 Design Snippet

The onCellEdit script shows how a single client-side rule addresses the list-edit problem.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);
}
```

5. Expected Fit Outcomes

Measure	Expected Result
Completeness	High-impact incidents always have an assignment group and assignee before saving.
Consistency	Urgency matches Impact for high-impact incidents.
Control	State changes happen only through the Incident form.
Flexibility	Restrictions are removed when Impact is no longer High.
User confidence	Clear messages guide users, reducing rework and uncertainty.

6. Conclusion

The analysis shows a strong fit between the problem and the solution. Users need help entering complete and consistent Incident data at the moment they work on the form. UI Policies and Client Scripts provide that help immediately, with little effort and little disruption to normal incidents.